Parallel I/O

Lecture 21

April 1, 2026

Recap
Write to Different Files

Independent writes

```c
#include "mpi.h"
#include <stdio.h>
#define BUFSIZE 1000

int main(int argc, char *argv[]) {

        int i, myrank, buf[BUFSIZE];
        char filename[128];
        FILE *myfile;

        MPI_Init(&argc, &argv);
        MPI_Comm_rank(MPI_COMM_WORLD, &myrank);

        sprintf(filename, "testfile.%d", myrank);
        myfile = fopen(filename, "w");

        for (i=0; i<BUFSIZE; i++) {
           buf[i] = myrank + i;
           fprintf(myfile, "%d ", buf[i]);
        }
        fprintf(myfile, "\n");
        fclose(myfile);

        MPI_Finalize();
        return 0;

}
~
```

Simple Parallel I/O Code – Shared File Recap

MPI_File fh

file_size_per_proc = FILESIZE / nprocs

MPI_File_open (MPI_COMM_WORLD, “/scratch/largefile”, MPI_MODE_RDONLY, MPI_INFO_NULL, &fh)

Returns file handle

MPI_File_seek (fh, rank*file_size_per_proc, MPI_SEEK_SET)

MPI_File_read (fh, buffer, count, MPI_INT, status)

MPI_File_close (&fh)

Parallel Read using Explicit Offset

MPI_Offset offset = (MPI_Offset) rank*file_size_per_proc*sizeof(int)

MPI_File_open (MPI_COMM_WORLD, “/scratch/largefile”, MPI_MODE_RDONLY, MPI_INFO_NULL, &fh)

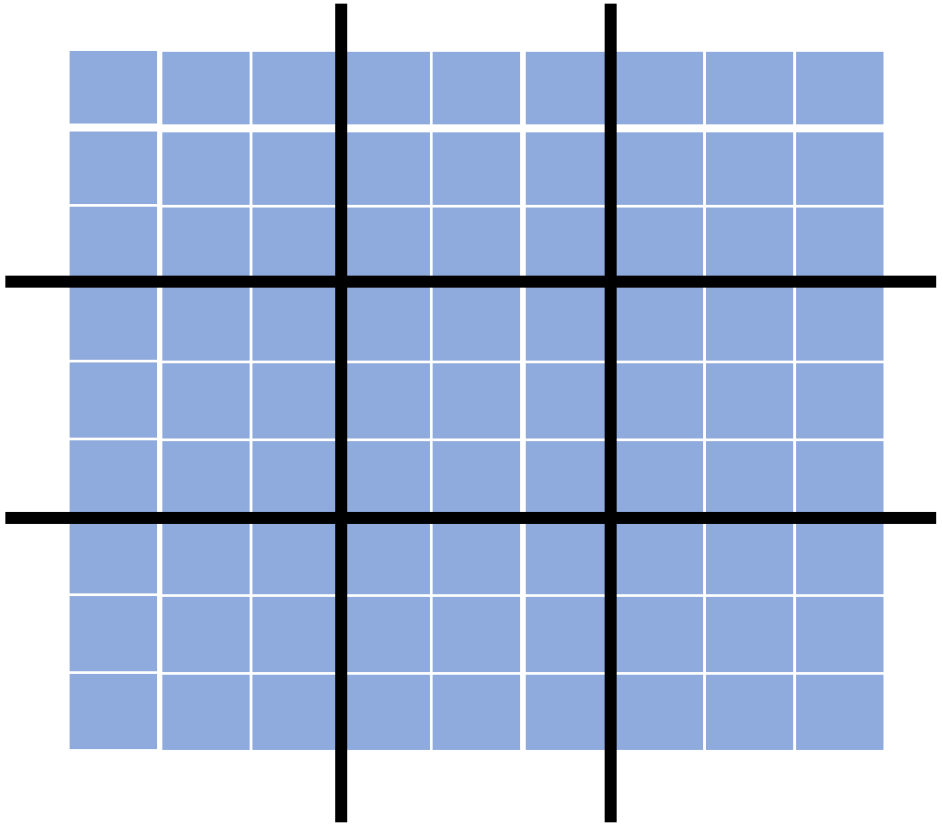
MPI_File_read_at (fh, offset, buffer, count, MPI_INT, status)

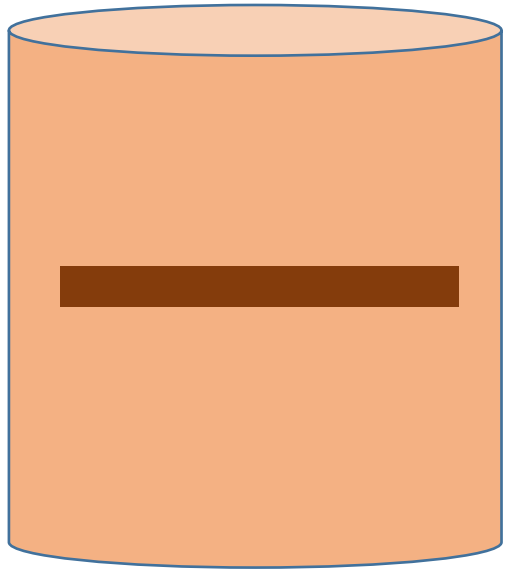
MPI_File_close (&fh)

```
MPI_File fh;  // FILE

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

for (int i=0; i<BUFSIZE ; i++)
  buf[i]=i;

strcpy(filename, "testfileIO");

// File open, fh: individual file pointer
MPI_File_open (MPI_COMM_WORLD, filename, MPI_MODE_CREATE | MPI_MODE_RDWR, MPI_INFO_NULL, &fh);

MPI_Offset fo = (MPI_Offset) myrank*BUFSIZE*sizeof(int);

// File write using explicit offset (independent I/O)
MPI_File_write_at (fh, fo, buf, BUFSIZE, MPI_INT, MPI_STATUS_IGNORE); //fwrite

// File read using explicit offset (independent I/O)
MPI_File_read_at (fh, fo, rbuf, BUFSIZE, MPI_INT, &status);   //fread

MPI_File_close (&fh);          //fclose

for (i=0; i<BUFSIZE ; i++)
  if (buf[i] != rbuf[i]) printf ("Mismatch [%d] %d %d\n", i, buf[i], rbuf[i]);
```

This will work for 1D domain, contiguous data writes per rank

Large Domain

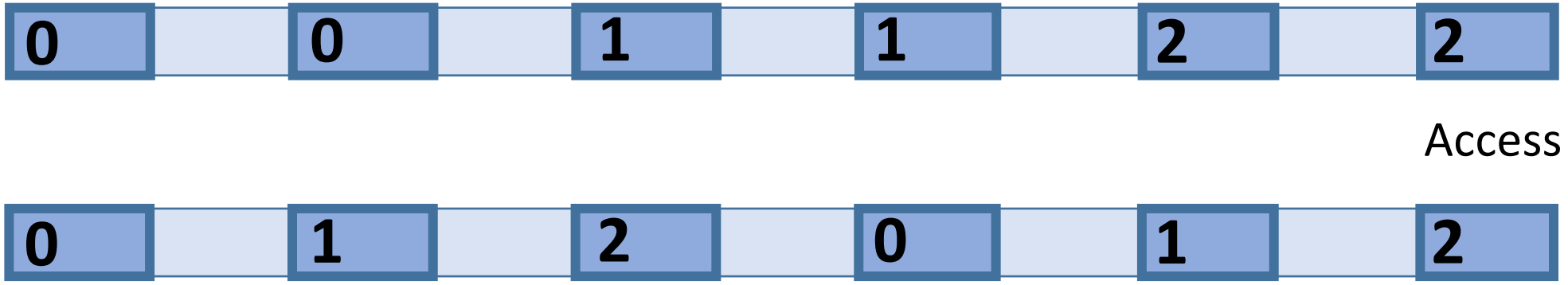




Access pattern

Non-contiguous Accesses

Example?



Frequently occurring file access pattern (non-contiguous)

What would be the required function calls?

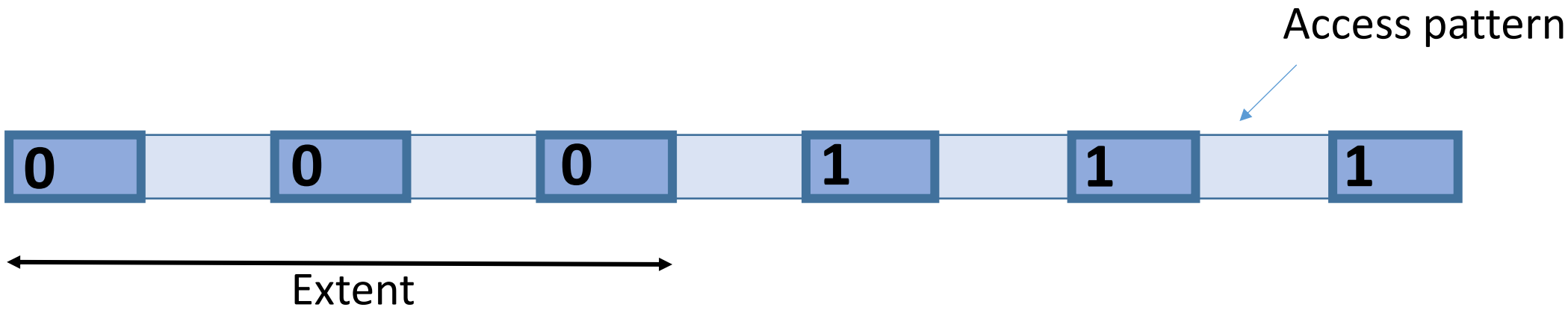

```
MPI_File fh;  // FILE

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

for (int i=0; i<BUFSIZE ; i++)
  buf[i]=i;

strcpy(filename, "testfileIO");

// File open, fh: individual file pointer
MPI_File_open (MPI_COMM_WORLD, filename, MPI_MODE_CREATE | MPI_MODE_RDWR, MPI_INFO_NULL, &fh);

MPI_Offset fo = (MPI_Offset) myrank*BUFSIZE*sizeof(int);

// File write using explicit offset (independent I/O)
MPI_File_write_at (fh, fo, buf, BUFSIZE, MPI_INT, MPI_STATUS_IGNORE); //fwrite

// File read using explicit offset (independent I/O)
MPI_File_read_at (fh, fo, rbuf, BUFSIZE, MPI_INT, &status);   //fread

MPI_File_close (&fh);          //fclose

for (i=0; i<BUFSIZE ; i++)
  if (buf[i] != rbuf[i]) printf ("Mismatch [%d] %d %d\n", i, buf[i], rbuf[i]);
```

What modification is required?

Offset modification required

What about number of calls?

Multiple Short Accesses



MPI_File_read_at (fh, offset1, buffer1, count1, MPI_INT, status)

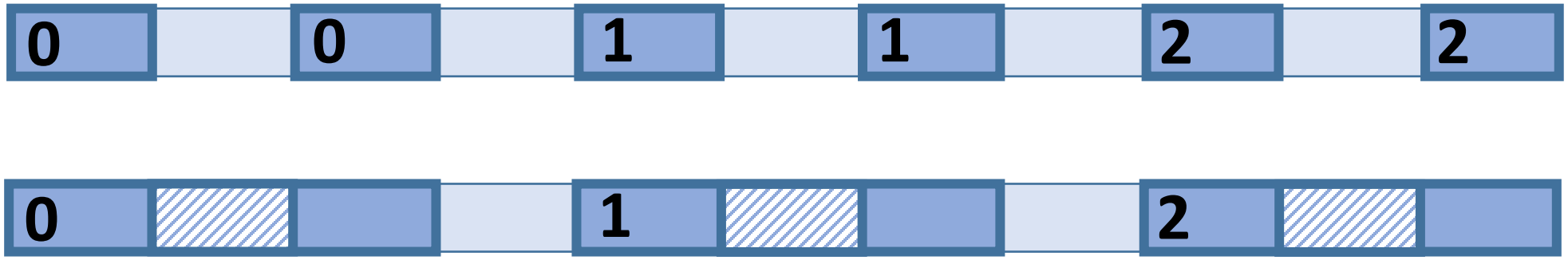
MPI_File_read_at (fh, offset2, buffer2, count2, MPI_INT, status)

MPI_File_read_at (fh, offset3, buffer3, count3, MPI_INT, status)

What is the problem with non-contiguous accesses?

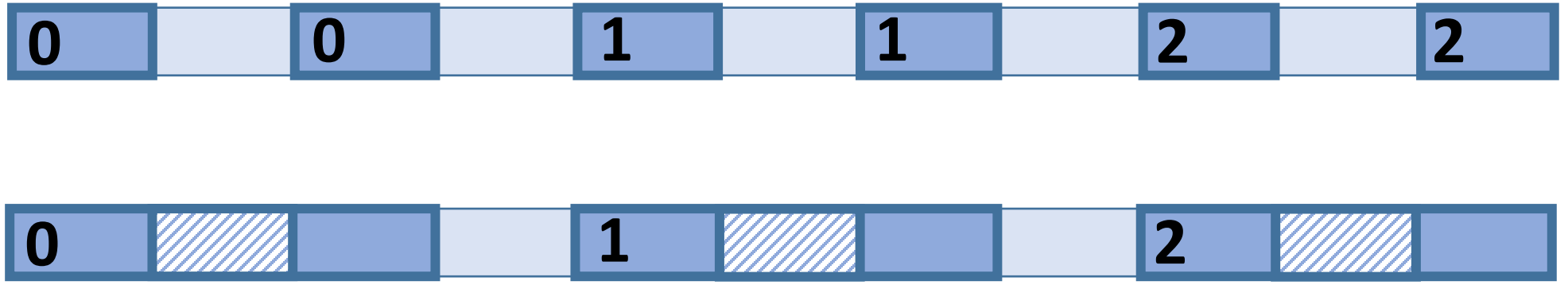
How can we optimize to issue only one read call?

Optimization – Data Sieving



- Goal: Avoid multiple I/O requests (costlier to issue reads/writes multiple times)
- Make large I/O requests and extract the data that is really needed
  - MPI library combines several reads to one read (start to end for each process) for every process
- Huge benefit of reading large, contiguous chunks

Data Sieving for Writes



- Copy only the user-modified data into the write buffer
- Write only the data that was modified – read-modify-write

User-controlled Independent MPI-IO Parameters

- ind_rd_buffer_size - Buffer size for data sieving for read
- ind_wr_buffer_size - Buffer size for data sieving for write
- romio_ds_read - Enable or not data sieving for read
- romio_ds_write - Enable or not data sieving for write

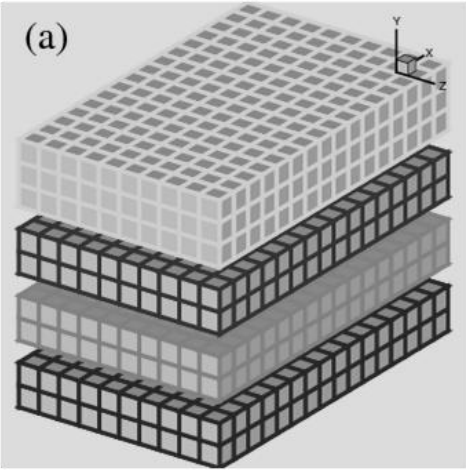
The data size per write
matters due to disk block sizes

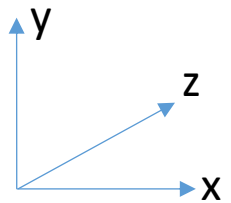
MPI_Info – Example

```
MPI_Info_create (&info);
MPI_Info_set (info, "ind_rd_buffer_size", "2097152");
MPI_Info_set (info, "ind_wr_buffer_size", "1048576");

MPI_File_open (MPI_COMM_WORLD, filename, amode, info, &fh);
```

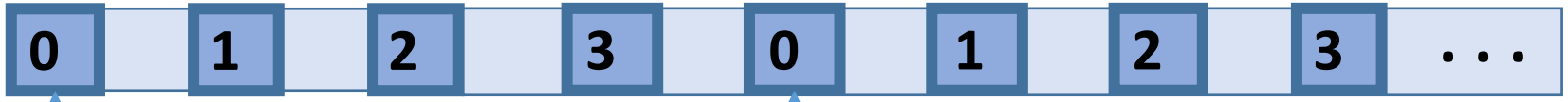
Non-contiguous I/O (3D Domain)

3D Domain Extent: N x N x N
Number of processes: P
1D decomposition along Y-axis
Sub-domain owned by each process: N x N/P x N





Recap (Lecture 17)



The first z-slice (z=0) owned by rank 0 (N x N/P)

The second z-slice (z=1) owned by rank 0 (N x N/P)

Data is written in XYZ order